

AI Model Deployment & Monitoring

Deployment Documentation · Version 1.0.0 · May 2026

Abstract

This report documents the end-to-end deployment of a RandomForestClassifier trained on the UCI Iris dataset as a production-ready REST API service. The project demonstrates containerisation with Docker, API design with FastAPI, structured JSON logging, in-process operational metrics, and health-check mechanisms. The documentation covers configuration details, endpoint specifications, monitoring strategies, and best practices drawn from industry standards for maintaining ML services in production.

1. The Significance of AI Deployment Practices

Developing an accurate machine-learning model is only the first step in delivering value from artificial intelligence. The model must be packaged, deployed, and maintained in an environment where real users or downstream systems can consume its predictions reliably, safely, and at scale. The gap between an experimental notebook and a production service involves a substantial set of engineering concerns that, if neglected, can undermine even the most sophisticated model.

Industry research consistently shows that the majority of ML projects fail to move from prototype to production. The barriers are rarely algorithmic — they are operational: poor reproducibility, lack of monitoring, inadequate error handling, and the absence of systematic deployment pipelines. Deployment engineering addresses these barriers by turning a trained artefact into a managed, observable service.

1.1 Reproducibility and Consistency

A model trained on a data scientist's laptop may behave differently in a cloud environment due to library version mismatches, operating-system differences, or environment variables. Containerisation with Docker eliminates this class of defect by bundling the entire runtime — interpreter, libraries, model artefact — into a single immutable image. The Dockerfile in this project uses a multi-stage build: a builder stage that compiles dependencies and a lean runtime stage that excludes build tools, minimising the attack surface and image size.

1.2 Scalability and Availability

A containerised service can be horizontally scaled by running multiple replicas behind a load balancer. Orchestration platforms such as Kubernetes or AWS ECS manage replica counts, rolling updates, and automatic restart on failure. The REST API design in this project is stateless — each request carries all the information required for a prediction — which is a prerequisite for seamless horizontal scaling.

1.3 Security and Governance

Production ML services handle sensitive data and must be auditable. The Dockerfile runs the application as a non-root user (UID 1001), reducing the blast radius of a container escape. Structured JSON logging ensures that every request — including the input features and predicted class — is captured in a machine-readable format suitable for audit trails and GDPR compliance workflows.

1.4 Continuous Monitoring and Model Health

A deployed model is subject to two classes of degradation. Technical degradation occurs when the serving infrastructure fails — elevated error rates, increased latency, or memory exhaustion. Statistical degradation (often called data drift or concept drift) occurs when the statistical properties of incoming data diverge from the training distribution, causing prediction quality to erode silently. Monitoring must address both dimensions. This project implements infrastructure monitoring (request counts, latency percentiles, error rates) as a foundation, and describes how statistical monitoring can be layered on top.

2. Container Configuration

The containerisation strategy employs Docker with a two-stage build pattern. The first stage (builder) installs all Python dependencies using pip with the `--prefix` flag to isolate installed packages. The second stage (runtime) starts from a fresh `python:3.11-slim` base and copies only the installed packages and application code, discarding the gcc toolchain and other build-time dependencies.

2.1 Dockerfile Walkthrough

```
FROM python:3.11-slim AS builder
WORKDIR /build
RUN apt-get update && apt-get install -y gcc \
  && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim AS runtime
RUN groupadd --gid 1001 appgroup && \
  useradd --uid 1001 --gid appgroup appuser
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app/ ./app/ | COPY model/ ./model/
RUN mkdir -p logs && chown -R appuser:appgroup /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
```

```
CMD python -c "import urllib.request; \
urllib.request.urlopen('http://localhost:8000/health')
```

Key decisions: (a) **python:3.11-slim** reduces the base image size by approximately 80 % compared to the full Debian image; (b) **--no-cache-dir** prevents pip from writing a local wheel cache that would inflate the image; (c) the **HEALTHCHECK** instruction causes Docker to mark the container as unhealthy if the /health endpoint does not respond within 10 seconds, enabling automatic replacement in orchestrated environments.

2.2 Docker Compose Configuration

The docker-compose.yml file defines the service, maps port 8000, mounts the host ./logs directory into the container at /app/logs so that structured log files survive container restarts, and sets resource limits (1 CPU core, 512 MB RAM). Resource limits prevent a single runaway service from starving co-located workloads on the host.

3. API Endpoint Reference

The API is built with FastAPI, which provides automatic OpenAPI documentation at /docs (Swagger UI) and /redoc. All requests and responses use JSON. Input validation is handled by Pydantic v2, which returns RFC 7807-style 422 Unprocessable Entity responses when validation fails.

Method	Path	Description	Auth
GET	/health	Liveness & readiness probe	None
GET	/metrics	Operational metrics snapshot	None
GET	/model/info	Model training metadata	None
POST	/predict	Single-sample Iris prediction	None
POST	/predict/batch	Batch prediction (1–256 samples)	None
GET	/docs	Swagger UI (interactive)	None
GET	/redoc	ReDoc documentation	None

3.1 POST /predict — Request & Response

Request body (JSON):

```
{ "sepal_length": 5.1, "sepal_width": 3.5,
  "petal_length": 1.4, "petal_width": 0.2 }
```

Successful response (HTTP 200):

```
{ "prediction": 0, "class_name": "setosa",
  "confidence": 1.0,
  "probabilities": {"setosa":1.0,"versicolor":0.0,"virginica":0.0} }
```

3.2 Request-ID Middleware

Every request receives a UUID4 identifier attached to the X-Request-ID response header and logged with every log line associated with that request. This enables end-to-end traceability across distributed log aggregation systems such as Elasticsearch or Google Cloud Logging.

4. Logging and Monitoring Implementation

4.1 Structured JSON Logging

The application uses Python's built-in logging module extended by `python-json-logger` to emit every log record as a single-line JSON object. Each record includes: timestamp (ISO 8601), logger name, level, message, service name (static: `iris-prediction-api`), and arbitrary key-value fields supplied by the caller. Structured logs are machine-parseable, eliminating the need for brittle regex-based log parsing in downstream analytics pipelines.

Example log record (formatted for readability):

```
{ "asctime": "2026-05-25T17:42:01.123Z",
  "name": "app.main", "level": "INFO",
  "message": "Prediction success",
  "service": "iris-prediction-api",
  "request_id": "a1b2c3d4-...",
  "prediction": "setosa", "confidence": 1.0,
  "latency_ms": 2.41 }
```

4.2 Rotating File Handler

In addition to the stdout console handler (for Docker log drivers), a `RotatingFileHandler` writes logs to `logs/api.log` with a maximum file size of 10 MB and up to 5 backup files (`api.log.1 ... api.log.5`). When the active file exceeds 10 MB, it is renamed and a new file is opened. This prevents unbounded disk consumption without losing recent history.

4.3 In-Process Operational Metrics

The `MetricsStore` class (`app/monitoring.py`) maintains thread-safe counters for `total_requests`, `successful_predictions`, `failed_requests`, and per-class prediction counts. A rolling deque of the last 1,000 latency measurements (in milliseconds) enables computation of the average and 95th-percentile latency without unbounded memory growth. All counters are protected by a `threading.Lock` to prevent race conditions under concurrent requests.

Metric	Description	Alerting Threshold (example)
<code>total_requests</code>	Cumulative request count	—
<code>successful_predictions</code>	Requests returning HTTP 200	—
<code>failed_requests</code>	Requests returning non-2xx	> 1 % of total
<code>avg_latency_ms</code>	Rolling mean latency	> 100 ms
<code>p95_latency_ms</code>	95th-percentile latency	> 500 ms
<code>error_rate_percent</code>	$\text{failed} / \text{total} \times 100$	> 5 %

requests_per_class	Per-class prediction distribution	Skew > 3σ
--------------------	-----------------------------------	------------------

5. Health Check Best Practices

Health checks are a fundamental primitive of cloud-native deployments. Kubernetes, Docker Swarm, AWS ECS, and most other orchestrators use health probes to determine whether a replica should receive traffic and whether it should be automatically replaced.

5.1 Liveness vs. Readiness

A **liveness probe** answers the question: 'Is the process alive?' If the probe fails, the container is killed and restarted. It should only fail for unrecoverable conditions — a deadlock, an OOM-killed subprocess, or a corrupted global state. A **readiness probe** answers: 'Is the service ready to accept traffic?' It may fail transiently while the model is loading or while a dependency (database, feature store) is unavailable. Failing the readiness probe removes the pod from the load-balancer rotation without restarting it.

The `/health` endpoint in this project combines both concerns into a single response. In a Kubernetes deployment manifest, the same path can be referenced for both `livenessProbe` and `readinessProbe`; the distinction is made by configuring different failure thresholds for each probe type.

5.2 Health Check Parameters

Parameter	Docker Compose Value	Kubernetes Equivalent
<code>initialDelaySeconds</code>	15 s (<code>start_period</code>)	<code>initialDelaySeconds: 15</code>
<code>periodSeconds</code>	30 s (<code>interval</code>)	<code>periodSeconds: 30</code>
<code>timeoutSeconds</code>	10 s (<code>timeout</code>)	<code>timeoutSeconds: 10</code>
<code>failureThreshold</code>	3 (<code>retries</code>)	<code>failureThreshold: 3</code>

5.3 Deep Health Checks

A shallow health check merely verifies that the HTTP server is accepting connections. A deep health check also validates that the model is loaded, that critical file paths are accessible, and optionally that a canary prediction on a known input returns the expected output. This project's `/health` endpoint currently implements a mid-depth check: it confirms that the model object is in memory. In a production setting, the canary prediction check can be added with a performance budget of a few milliseconds.

6. Handling Unexpected Behaviour and Performance Drift

6.1 Input Validation as the First Line of Defence

Pydantic schemas enforce type correctness and value ranges at the API boundary before any inference code is reached. Fields with out-of-range values or wrong types return a 422 response with a detailed error message. This prevents obviously malformed inputs from consuming model resources and polluting performance metrics.

6.2 Data Drift Detection

Data drift occurs when the distribution of incoming feature values shifts away from the training distribution. In a production system, this can be detected by logging feature values with each request and running a statistical test (Kolmogorov–Smirnov, Population Stability Index, or Jensen–Shannon divergence) on a rolling window of recent requests compared to the training distribution. Tools such as Evidently AI, WhyLogs, or Amazon SageMaker Model Monitor provide managed implementations of this pattern.

6.3 Concept Drift and Retraining Triggers

Even when input distributions are stable, the relationship between inputs and the correct output may change (concept drift). Detecting concept drift requires ground truth labels for recent predictions, which are not always available in real time. Common strategies include: (a) shadow deployment — running the new and old models in parallel and comparing outputs; (b) canary release — routing a small percentage of traffic to the new model and monitoring error rates; (c) champion/ challenger testing — periodically retraining on recent data and promoting the challenger if it outperforms the champion on a held-out evaluation set.

6.4 Circuit Breaker Pattern

When the model service is overloaded or a downstream dependency fails, the circuit breaker pattern prevents cascading failures by returning a fast error to callers rather than queuing requests until timeouts occur. In Python, the `tenacity` or `pybreaker` libraries implement this pattern. The `/health` endpoint can expose circuit-breaker state so that load balancers can shed traffic proactively.

7. Maintenance of Deployed AI Services

7.1 Versioning Strategy

The API version (1.0.0) is embedded in the Docker image tag and returned by the `/health` endpoint. When a new model version is released, the image is rebuilt with an incremented tag. Rolling updates in Kubernetes replace old pods with new ones without downtime, using the `RollingUpdate` deployment strategy with a `maxSurge` of 1 and `maxUnavailable` of 0.

7.2 Model Artefact Management

The `iris_model.joblib` artefact is baked into the Docker image at build time, ensuring that the image is fully self-contained and deployable without external dependencies. For larger models or frequent retraining cycles, the artefact should be stored in object storage (S3, GCS) and fetched at container startup, with the artefact URI passed as an environment variable.

7.3 Log Aggregation and Alerting

Structured JSON logs emitted to stdout are captured by the Docker log driver (json-file by default). In production, a log-shipper sidecar (Fluent Bit, Filebeat) forwards logs to a central store (Elasticsearch, Loki, Cloud Logging). Dashboards in Grafana or Kibana visualise request volume, error rates, and latency percentiles. Alert rules fire when `error_rate_percent` exceeds 5 % or `p95_latency_ms` exceeds 500 ms for a sustained window.

7.4 Dependency Patching

The base image (python:3.11-slim) should be rebuilt regularly to incorporate OS security patches. Dependabot or Renovate can automate dependency version updates in `requirements.txt`, and a CI pipeline can rebuild and re-test the image on every pull request. Container image scanning tools (Trivy, Snyk, Gripe) should be integrated into the CI pipeline to detect known CVEs before an image is promoted to production.

7.5 Capacity Planning

The Iris classifier is a lightweight sklearn model; a single CPU-bound worker can handle thousands of requests per second at sub-millisecond latency. For larger deep-learning models, capacity planning must account for GPU availability, batch size, and model warm-up time. Horizontal Pod Autoscaler (HPA) in Kubernetes scales the number of replicas based on CPU utilisation or custom metrics exposed by the `/metrics` endpoint.

8. Project Summary and File Reference

File / Directory	Purpose
model/train_and_save.py	Trains RandomForest on Iris data; saves iris_model.joblib + metadata
app/main.py	FastAPI app: endpoints, middleware, lifespan model loading
app/model.py	Singleton model loader and predict_single / predict_batch functions
app/monitoring.py	Thread-safe MetricsStore: counters + rolling latency deque
app/logger_config.py	JSON logger with rotating file handler
app/schemas.py	Pydantic v2 request/response schemas with field validation
tests/test_api.py	pytest suite covering all endpoints
Dockerfile	Multi-stage build; non-root user; HEALTHCHECK
docker-compose.yml	Service definition; volume mounts; resource limits
requirements.txt	Pinned Python dependencies
generate_report.py	Generates this PDF using reportlab

9. Conclusion

This project demonstrates that transforming a machine-learning model from an experiment into a maintainable production service requires deliberate engineering investment across containerisation, API design, structured observability, and health management. The Iris classifier example is intentionally simple, allowing attention to focus on the deployment infrastructure rather than model complexity. The patterns established here — immutable container images, stateless REST APIs, structured JSON logs, rolling metric counters, and health endpoints — scale directly to production ML systems serving millions of requests per day.

Future work for a production hardening of this system would include: (1) integration with Prometheus and Grafana for time-series metrics and alerting dashboards; (2) OpenTelemetry distributed tracing across microservice boundaries; (3) automated retraining pipelines triggered by data-drift detection; (4) A/B testing infrastructure for safe model upgrades; and (5) formal SLO definitions (e.g., 99.9 % availability, p99 latency < 200 ms) with automated SLO burn-rate alerts.